

Document number: P1202R5

Date: 2022-11-10

Reply-to: David Goldblatt <davidtgoldblatt@gmail.com>

Audience: LWG

P1202R5: Asymmetric Fences

Background

Some types of concurrent algorithms can be split into a common path and an uncommon path, both of which require fences (or other operations with non-relaxed memory orders) for correctness. On many platforms, it's possible to speed up the common path by adding an even stronger fence type (stronger than `memory_order::seq_cst`) down the uncommon path. These facilities are being used in an increasing number of concurrency libraries. We propose standardizing these asymmetric fences, and incorporating them into the memory model.

The proposed ship vehicle is Concurrency TS 2.

In Kona, LWG asked for a number of wording clarifications to the R4 version of this paper; this version updates the wording with those clarifications. In the interest of brevity, this omits much of the context that already has directional approval; see P1202R0 for an in-depth description of the technique and its uses, and P1202R1 for an argument that this is the “right” memory-model-ese for this technique.

Wording

As a diff for the TS to apply to the IS:

33.5.4 Order and consistency [atomics.order]

In subclause 33.5.4 [atomics.order], strike the word “four” in the phrase “the following four conditions are required to be satisfied by S:” and add the following two bullets to the list:

- if a `memory_order::seq_cst` lightweight-fence X happens before A and B happens before a `memory_order::seq_cst` heavyweight-fence Y, then X precedes Y in S; and
- if a `memory_order::seq_cst` heavyweight-fence X happens before A and B happens before a `memory_order::seq_cst` lightweight-fence Y, then X precedes Y in S.

And, as a pure insertion, with a section number to be filled in by the editor:

X.Y Header <experimental/asymmetric_fence> synopsis

Add the following declarations to the synopsis of the header <experimental/asymmetric_fence>:

```

namespace std::experimental::inline concurrency_v2 {
    // 3.2.1 asymmetric_thread_fence_heavy
    void asymmetric_thread_fence_heavy(memory_order order) noexcept;
    // 3.2.2 asymmetric_thread_fence_light
    void asymmetric_thread_fence_light(memory_order order) noexcept;
}

```

X.Z Asymmetric fences [atomics.fences.asym]

This subclause introduces synchronization primitives called *heavyweight-fences* and *lightweight-fences*. Like fences, heavyweight-fences and lightweight-fences can have acquire semantics, release semantics, or both, and can be sequentially consistent (in which case they are included in the total order `S` on `memory_order::seq_cst` operations). A heavyweight-fence with acquire semantics is called an *acquire heavyweight-fence*. A heavyweight-fence has all the synchronization effects of a fence (33.5.11 [atomics.fences]). [Note: Heavyweight-fences and lightweight-fences are distinct from fences. -- end note]

A heavyweight-fence with acquire semantics is called an *acquire heavyweight-fence*. A heavyweight-fence with release semantics is called a *release heavyweight-fence*. A lightweight-fence with acquire semantics is called an *acquire lightweight-fence*. A lightweight-fence with release semantics is called a *release lightweight-fence*.

If there are evaluations `A` and `B`, and atomic operations `X` and `Y`, both operating on some atomic object `M`, such that `A` is sequenced before `X`, `X` modifies `M`, `Y` is sequenced before `B`, and `Y` reads the value written by `X` or a value written by any side effect in the hypothetical release sequence `X` would head if it were a release operation, and one of the following hold:

- `A` is a release lightweight-fence and `B` is an acquire heavyweight-fence; or
- `A` is a release heavyweight-fence and `B` is an acquire lightweight-fence

then any evaluation sequenced before `A` strongly happens before any evaluation that `B` is sequenced before.

```

void asymmetric_thread_fence_heavy(memory_order order) noexcept;

```

1. Effects: Depending on the value of `order`, this operation:

- has no effects, if `order == memory_order::relaxed`;
- is an acquire heavyweight-fence, if `order == memory_order::acquire` or `order == memory_order::consume`;
- is a release heavyweight-fence, if `order == memory_order::release`;
- is both an acquire heavyweight-fence and a release heavyweight-fence, if `order == memory_order::acq_rel`;
- is a sequentially consistent acquire and release heavyweight-fence, if `order == memory_order::seq_cst`.

```
void asymmetric_thread_fence_light(memory_order order) noexcept;
```

1. Effects: Depending on the value of order, this operation:

- has no effects, if order == memory_order::relaxed;
- is an acquire lightweight-fence, if order == memory_order::acquire or order == memory_order::consume;
- is a release lightweight-fence, if order == memory_order::release;
- is both an acquire lightweight-fence and a release lightweight-fence, if order == memory_order::acq_rel;
- is a sequentially consistent acquire and release lightweight-fence, if order == memory_order::seq_cst.

[Note: Delegating both heavyweight-fence and lightweight-fence functions to an atomic_thread_fence(order) call is a valid implementation. Implementations can adopt techniques in which calls to asymmetric_thread_fence_light execute more quickly than calls to atomic_thread_fence with the same memory_order, at the cost of asymmetric_thread_fence_heavy executing more slowly than calls to atomic_thread_fence with the same memory_order.]

Add a feature test macro in <experimental/asymmetric_thread_fence>:

```
#define __cpp_lib_experimental_asymmetric_fence 202XYZ
```

Add the following entry into the feature test macro table inserted via P2396R1

Title	Subclause	Macro name	Value	Header
Asymmetric Fences	X.Y	__cpp_lib_experimental_asymmetric_fence	2022XY	<experimental/asymmetric_thread_fence>